Thompson Sampling for Finite-Budget Backgammon Decision Problems

The Basic Problem

We study a **single local backgammon decision problem**:

- one board state
- one realized dice roll
- the legal moves from that state and roll
- a fixed simulation budget
- a declared continuation-play / rollout policy

For each legal move i, there is an unknown rollout-model value

$$\mu_i = \mathbb{E}[Y_i].$$

Core question

Given a fixed rollout budget, how should simulations be allocated across legal moves so that the best move, or the ranking of moves, is identified as accurately and efficiently as possible?

Why This Is Difficult

The problem is difficult for several reasons:

1. **Outcomes are stochastic.** The same move can lead to different future games.
2. **Budgets are finite.** We cannot evaluate every move exhaustively.
3. **Move values may be close.** The best and second-best moves may differ only slightly.
4. **Outcome distributions differ by move.** Some moves are stable, others are high-variance or asymmetric.

So the problem is not just "estimate all move values." It is also:

decide where each additional rollout should go.

Proxy Truth

In this package, "truth" does **not** mean exact backgammon truth, expert truth, or game-theoretic truth.
Instead, proxy truth is:

high-budget Monte Carlo reference under the same rollout environment.

This means:

- the same continuation-play policy
- the same reward mapping
- the same rollout environment
- a much larger budget than the online allocation method receives

So we evaluate methods against a stable **Monte Carlo proxy reference**, not against an external oracle.

Bayesian View of the Problem

For each move i, the unknown quantity of interest is its value μ_i.
Bayesianly, we treat μ_i or a more detailed parameter θ_i as random and update beliefs after observing rollouts:

$$p(\theta_i \mid D_i) \propto p(D_i \mid \theta_i)\, p(\theta_i).$$

The algorithm repeatedly:

1. observes rollout outcomes,
2. updates the posterior for each move,
3. uses the posterior to decide which move to simulate next.

Thompson sampling is one way of converting posterior uncertainty into sequential decisions.

General Thompson Sampling Template

At each step t, for each move i, we maintain a posterior distribution:

$$p(\theta_i \mid D_{i,t}).$$

We then:

1. draw a plausible parameter value $\tilde{\theta}_i$ from the posterior,
2. convert it into a sampled move value $\tilde{\mu}_i$,
3. choose the move with the highest sampled value.

Formally,

$$\tilde{\theta}_i \sim p(\theta_i \mid D_{i,t}), \qquad \tilde{\mu}_i = g(\tilde{\theta}_i), \qquad a_t = \arg\max_i \tilde{\mu}_i.$$

The specific form of g depends on the model.

Three Bayesian Models Used in This Project

We focus on three conceptually clean model classes:

1. **Beta–Bernoulli**
   - reward is binary: win / loss
   - target: probability of winning
2. **Dirichlet–Categorical**
   - reward is a discrete scored outcome category
   - target: full outcome distribution, then expected score
3. **Student-t scalar payoff**
   - reward is a scalar payoff
   - target: expected payoff with uncertainty in variance

These models differ in what they regard as the basic rollout outcome.

Beta–Bernoulli: What It Models

Suppose each rollout outcome is reduced to:

$$Y_{ij} \in \{0, 1\},$$

where:

- $1 =$ the move eventually wins,
- $0 =$ the move eventually loses.

For move i, let p_i be the probability of winning. Then:

$$Y_{ij} \mid p_i \sim \text{Bernoulli}(p_i).$$

This is the cleanest model if the scientific target is:

which move maximizes win probability?

Beta–Bernoulli: Prior, Posterior, and TS Draw

Prior

$$p_i \sim \mathrm{Beta}(a_0, b_0).$$

Likelihood

$$Y_{ij} \mid p_i \sim \mathrm{Bernoulli}(p_i).$$

After observing w_i wins and ℓ_i losses:

Posterior

$$p_i \mid D_i \sim \mathrm{Beta}(a_0 + w_i,\ b_0 + \ell_i).$$

Thompson step

$$\tilde{p}_i \sim \mathrm{Beta}(a_0 + w_i,\ b_0 + \ell_i),$$

and the sampled move value is just $\tilde{p}_i$.

Dirichlet–Categorical: What It Models

Here the rollout outcome is a scored category rather than a binary event.
A natural backgammon outcome space is:

{single loss, gammon loss, backgammon loss, single win, gammon win, backgammon win}.

For move i, let

$$\theta_i = (\theta_{i1}, \ldots, \theta_{iK})$$

be the probabilities of these K categories. Then:

$$Y_{ij} \mid \theta_i \sim \mathrm{Categorical}(\theta_i).$$

This is the most natural model if the scientific target is:

which move maximizes expected backgammon score?

Dirichlet–Categorical: Prior, Posterior, and TS Draw

Prior

$$\theta_i \sim \mathrm{Dirichlet}(\alpha_{i1}, \ldots, \alpha_{iK}).$$

Likelihood

$$Y_{ij} \mid \theta_i \sim \mathrm{Categorical}(\theta_i).$$

If the observed category counts for move i are $c_{i1}, \ldots, c_{iK}$, then:

$$\theta_i \mid D_i \sim \mathrm{Dirichlet}(\alpha_{i1} + c_{i1}, \ldots, \alpha_{iK} + c_{iK}).$$

If $v = (v_1, \ldots, v_K)$ is the payoff vector, then

$$\tilde{\theta}_i \sim \mathrm{Dirichlet}(\alpha_i + c_i), \qquad \tilde{\mu}_i = \sum_{k=1}^{K} v_k \tilde{\theta}_{ik}.$$

Student-t Scalar Model: What It Models

Suppose each rollout outcome is encoded as a scalar payoff:

$$Y_{ij} \in \{-3, -2, -1, +1, +2, +3\}$$

or an equivalent normalized score.
We model:

$$Y_{ij} \mid \mu_i, \sigma_i^2 \sim \mathcal{N}(\mu_i, \sigma_i^2).$$

This is not exact to the discrete scored outcome process, but it is a **proper Bayesian scalar model** for mean payoff.
It is useful when the scientific target is:

which move has the highest expected payoff?

Student-t Scalar Model: Prior and Posterior Updates

Use a Normal–Inverse-Gamma prior:

$$\mu_i \mid \sigma_i^2 \sim \mathcal{N}(m_0, \sigma_i^2/\kappa_0), \qquad \sigma_i^2 \sim \mathrm{InvGamma}(\alpha_0, \beta_0).$$

After n_i rollouts with sample mean $\bar{y}_i$ and centered sum of squares

$$S_i = \sum_{j=1}^{n_i} (y_{ij} - \bar{y}_i)^2,$$

the updated hyperparameters are:

$$\kappa_n = \kappa_0 + n_i, \qquad m_n = \frac{\kappa_0 m_0 + n_i \bar{y}_i}{\kappa_0 + n_i},$$

$$\alpha_n = \alpha_0 + \frac{n_i}{2}, \qquad \beta_n = \beta_0 + \frac{1}{2} S_i + \frac{\kappa_0 n_i}{2(\kappa_0 + n_i)} (\bar{y}_i - m_0)^2.$$

Student-t Scalar Model: Why the Posterior Mean is Student-t

Under the Normal–Inverse-Gamma prior, the posterior for (μ_i, σ_i^2) stays in the same family. If we integrate out the unknown variance σ_i^2, the marginal posterior for the mean becomes:

$$\mu_i \mid D_i \sim t_{2\alpha_n}\left(m_n, \sqrt{\frac{\beta_n}{\alpha_n \kappa_n}}\right).$$

This is important because:

- it gives heavier tails than a fixed-variance Gaussian posterior,
- it reflects uncertainty in both the mean and the variance,
- it is often more cautious early in the budget when data are scarce.

Thompson step

Draw $\tilde{\mu}_i$ from this posterior Student-t distribution and choose the move with the largest draw.

How the Algorithms Differ

All methods use the same broad ingredients:

- a posterior model for each move,
- sequential rollouts,
- repeated posterior updating.

What changes is:

how the next move is selected from the posterior information.

So the algorithms do *not* differ in what the data are. They differ in how aggressively they:

- explore uncertain moves,
- focus on top contenders,
- eliminate weak moves,
- protect against premature lock-in.

Standard Thompson Sampling: Why It Exists

Standard TS is the baseline posterior-sampling rule.
Its central idea is simple:

Key idea

Act as if one posterior draw were the truth.

Why this is attractive:

- If a move is uncertain, its posterior is wide, so it still has a chance to be sampled as best.
- If a move looks strong, it is sampled as best more often.
- No separate exploration bonus is required.

The method is simple, interpretable, and the natural starting point for all comparisons in this project.

Standard Thompson Sampling: Mathematical Interpretation

At time t, standard TS draws one posterior value for each move:

$$\tilde{\mu}_{i,t} \sim p(\mu_i \mid D_{i,t}).$$

It then chooses

$$a_t = \arg\max_i \tilde{\mu}_{i,t}.$$

A useful interpretation is that the selection probability of move i is

$$\mathbb{P}(a_t = i \mid D_t) = \mathbb{P}\left(\tilde{\mu}_{i,t} = \max_j \tilde{\mu}_{j,t} \mid D_t\right).$$

So standard TS samples each move in proportion to how often that move looks best under the current posterior.

Standard Thompson Sampling: Pseudocode

1: Initialize posterior state for each legal move
2: **for** $t = 1, \ldots, B$ **do**
3: Draw one posterior sample value $\tilde{\mu}_i$ for every move i
4: Choose $a_t \leftarrow \arg\max_i \tilde{\mu}_i$
5: Run one rollout from move a_t
6: Observe outcome Y_t
7: Update posterior state for move a_t
8: **end for**
9: Return final recommendation based on final posterior summaries

Top-Two Thompson Sampling: Why It Exists

A common criticism of standard TS in pure-exploration settings is that it may spend too much effort outside the main competition.
If the real difficulty is distinguishing the top few moves, then it can be more useful to repeatedly compare:

- the current posterior winner,
- a plausible challenger.

That is the motivation for TTTS.
TTTS is designed to focus more strongly on the region where the final decision is likely to be determined:

the competition among the leading moves.

Top-Two Thompson Sampling: How to Interpret It

At each step, TTTS first identifies a posterior winner exactly as standard TS would.
Then, instead of always sampling that winner, it sometimes forces a second draw until a *different* winner appears.
So the algorithm alternates between:

- reinforcing the current best-looking move,
- explicitly challenging it with another plausible contender.

This is why TTTS is often easier to justify in best-arm-identification settings than in cumulative-reward settings: it directly targets the top competition.

Top-Two Thompson Sampling: Mathematical Interpretation

Let

$$w_t = \arg\max_i \tilde{\mu}_{i,t}$$

be the first posterior winner.
With probability β, TTTS allocates to w_t. With probability $1-\beta$, it keeps drawing until a *different* winner appears, say $c_t \neq w_t$.
So, schematically,

$$a_t = \begin{cases} w_t, & \text{with probability } \beta, \\ c_t, & \text{with probability } 1-\beta. \end{cases}$$

This changes the allocation rule from "sample according to posterior-best probability" to "sample the winner, but often sample its strongest challenger instead." That is exactly why TTTS tends to spend more effort on the top competition.

Top-Two Thompson Sampling: Pseudocode

Initialize posterior state for each legal move
for $t = 1, \ldots, B$ **do**
    Draw one posterior sample value $\tilde{\mu}_i$ for every move i
    Let $w \leftarrow \arg\max_i \tilde{\mu}_i$ ▷ first posterior winner
    Draw $U \sim \mathrm{Uniform}(0, 1)$
    if $U \leq \beta$ **then**
        $a_t \leftarrow w$
    else
        repeat
            Draw a fresh posterior sample vector $\tilde{\mu}'_i$ for all moves
            Let $c \leftarrow \arg\max_i \tilde{\mu}'_i$
        until $c \neq w$
        $a_t \leftarrow c$ ▷ challenger
    end if
    Run one rollout from move a_t, observe Y_t, and update that posterior state
end for
Return final recommendation

Multi-Sample Thompson Sampling: Why It Exists

Standard TS uses only *one* posterior draw per move at each step. This makes it simple, but also noisy.
A natural variation is to ask:

> If I repeatedly sampled from the posterior, which move would most often look best?

That is the motivation for multi-sample TS. It tries to estimate posterior best-arm probability more explicitly by using several posterior worlds at once rather than one.

Multi-Sample Thompson Sampling: How to Interpret It

For each sequential decision, the method:

- draws many posterior samples,
- records which move wins in each posterior draw,
- chooses the move that wins most often.

So instead of asking:

Which move wins in one sampled world?

it asks:

Which move wins most often across many sampled worlds?

This typically makes the selection rule more stable and less sensitive to one lucky draw.

Multi-Sample Thompson Sampling: Mathematical Interpretation

Let M be the number of posterior worlds sampled at one decision step. For each world m define the winning move

$$w^{(m)} = \arg\max_i \tilde{\mu}_i^{(m)}.$$

Then define the empirical posterior-best frequency

$$\hat{p}_{i,t} = \frac{1}{M} \sum_{m=1}^{M} \mathbf{1}\{w^{(m)} = i\}.$$

The method selects

$$a_t = \arg\max_i \hat{p}_{i,t}.$$

So multi-sample TS is a Monte Carlo approximation to selecting the move with the largest posterior probability of being best.

Multi-Sample Thompson Sampling: Pseudocode

Initialize posterior state for each legal move
for $t = 1, \ldots, B$ **do**
    for each move i **do**
        Set $\texttt{winCount}[i] \leftarrow 0$
    end for
    for $m = 1, \ldots, M$ **do**
        Draw one posterior sample value $\tilde{\mu}_i^{(m)}$ for every move i
        Let $w_m \leftarrow \arg\max_i \tilde{\mu}_i^{(m)}$
        Increment $\texttt{winCount}[w_m]$
    end for
    Choose $a_t \leftarrow \arg\max_i \texttt{winCount}[i]$
    Run one rollout from move a_t
    Observe outcome Y_t
    Update posterior state for move a_t
end for
Return final recommendation

Soft Elimination Thompson Sampling: Why It Exists

One weakness of standard TS is that it may continue allocating budget to moves that are already very unlikely to be best.
The idea of soft elimination is:

If a move's posterior chance of being best becomes extremely small, stop spending precious rollouts on it.

This is especially attractive under fixed budgets, where every rollout spent on a hopeless move is one not spent resolving the top competition.

Soft Elimination Thompson Sampling: How to Interpret It

This method does *not* hard-code elimination from raw point estimates.
Instead, it uses the posterior:

- estimate the probability that each move is best,
- eliminate only when that probability is sufficiently small.

So it is still uncertainty-aware. The elimination is "soft" because it is driven by posterior evidence rather than a crude deterministic rule.
The practical interpretation is:

focus budget on plausible winners, not the full move set.

Soft Elimination Thompson Sampling: Mathematical Interpretation

Define an estimate of posterior-best probability,

$$\hat{p}_{i,t} \approx \mathbb{P}(i \text{ is best} \mid D_t),$$

for example by repeated posterior sampling.
Given a threshold τ, define the active set

$$\mathcal{A}_t = \{i : \hat{p}_{i,t} \geq \tau\}.$$

The method then performs Thompson-style allocation only within $\mathcal{A}_t$.
In practice, most implementations also impose a safeguard such as keeping at least a minimum number of active moves so that the method does not eliminate too aggressively.

Soft Elimination Thompson Sampling: Pseudocode

Initialize posterior state for each legal move
Set active set $\mathcal{A} \leftarrow$ all legal moves
for $t = 1, \ldots, B$ **do**
    Estimate $\mathbb{P}(i \text{ is best} \mid D_t)$ for each move $i \in \mathcal{A}$
    for each move $i \in \mathcal{A}$ **do**
        if $\mathbb{P}(i \text{ is best} \mid D_t) < \tau$ **then**
            Mark i for elimination
        end if
    end for
    Update active set $\mathcal{A}$, keeping a minimum number of moves active
    Draw one posterior sample value $\tilde{\mu}_i$ for every move $i \in \mathcal{A}$
    Choose $a_t \leftarrow \arg\max_{i \in \mathcal{A}} \tilde{\mu}_i$
    Run one rollout from move a_t, observe Y_t, and update that posterior state
end for
Return final recommendation

Forced Exploration Thompson Sampling: Why It Exists

Posterior methods can become too confident too early if initial rollouts are misleading. Forced exploration addresses this by saying:

Even if the posterior currently favors one move, occasionally check other moves on purpose.

This is not meant to replace Thompson sampling. It is a safeguard against premature lock-in and posterior myopia.

Forced Exploration Thompson Sampling: How to Interpret It

Most of the time, the algorithm behaves exactly like standard TS.
But with small probability ε, it overrides the Thompson choice and instead explores according to a simpler rule, such as:

- uniform random choice,
- least-sampled move.

So the interpretation is:

trust the posterior most of the time, but reserve a small exploration budget for safety.

Forced Exploration Thompson Sampling: Mathematical Interpretation

This method is a mixture rule. If a_t^{TS} is the standard Thompson choice and a_t^{explore} is an exploration choice, then

$$a_t = \begin{cases} a_t^{\mathrm{explore}}, & \text{with probability } \varepsilon, \\ a_t^{\mathrm{TS}}, & \text{with probability } 1-\varepsilon. \end{cases}$$

Equivalently, the induced action probabilities are a convex mixture of:

- the TS selection probabilities
- the exploration distribution

This means the method remains mostly Thompson-like, but cannot become completely deterministic too early.

Forced Exploration Thompson Sampling: Pseudocode

Initialize posterior state for each legal move
for $t = 1, \ldots, B$ **do**
    Draw $U \sim \mathrm{Uniform}(0, 1)$
    if $U \leq \varepsilon$ **then**
        Choose a_t by exploration rule ▷ for example: uniform or least-sampled
    else
        Draw one posterior sample value $\tilde{\mu}_i$ for every move i
        Choose $a_t \leftarrow \arg\max_i \tilde{\mu}_i$
    end if
    Run one rollout from move a_t
    Observe outcome Y_t
    Update posterior state for move a_t
end for
Return final recommendation

Top-k Focused Thompson Sampling: Why It Exists

In many local backgammon decisions, only a small subset of moves are serious contenders. So rather than sampling over the full action set indefinitely, we can say:

Core idea

Focus most of the budget on the currently most promising k moves.

This is appealing when the action set is moderate and the method's real task is to resolve the top competition rather than to refine the ranking of clearly weak moves.

Top-k Focused Thompson Sampling: How to Interpret It

At each stage, the method identifies a promising subset

$$\mathcal{K}_t = \{\text{current top } k \text{ moves}\}.$$

Then it applies Thompson-style allocation mostly inside $\mathcal{K}_t$.
Important caution:

- If k is too small
- and the early ranking is wrong,

then the true best move can be frozen out.
So a sensible implementation includes occasional exploration outside the current top-k set.

Top-k Focused Thompson Sampling: Mathematical Interpretation

Let $s_{i,t}$ be a score used to define promising moves, such as:

- posterior mean,
- estimated probability-best.

Define the current top-k set by

$$\mathcal{K}_t = \mathrm{TopK}(s_{1,t}, \ldots, s_{K,t}; k).$$

Then the method typically uses a mixture rule:

- with high probability, sample by TS inside $\mathcal{K}_t$;
- with small probability, explore outside $\mathcal{K}_t$.

So the method is trying to approximate the idea that only a small subset of moves is currently decision-relevant.

Top-k Focused Thompson Sampling: Pseudocode

Initialize posterior state for each legal move
for $t = 1, \ldots, B$ **do**
    Compute a current score s_i for every move i ▷ posterior mean or estimated probability-best
    Let $\mathcal{K}_t \leftarrow$ the current top-k moves by score
    Draw $U \sim \mathrm{Uniform}(0, 1)$
    if $U \leq \varepsilon_{\text{out}}$ **then**
        Choose a_t from outside $\mathcal{K}_t$
    else
        Draw one posterior sample value $\tilde{\mu}_i$ for every move $i \in \mathcal{K}_t$
        Choose $a_t \leftarrow \arg\max_{i \in \mathcal{K}_t} \tilde{\mu}_i$
    end if
    Run one rollout from move a_t
    Observe outcome Y_t
    Update posterior state for move a_t
end for
Return final recommendation

Equal Allocation: Why It Is Still Useful

Equal allocation is not a Thompson method, but it is an important baseline.
It answers:

Question

What happens if we ignore posterior information and simply spread the budget evenly?

Why include it?

- It is simple.
- It is easy to interpret.
- It clarifies how much adaptivity actually helps.

Equal Allocation: Pseudocode

1: Initialize sample count for each legal move
2: **for** $t = 1, \ldots, B$ **do**
3: Choose move a_t with smallest current sample count
4: Run one rollout from move a_t
5: Observe outcome Y_t
6: Update posterior state for move a_t
7: Increment sample count for move a_t
8: **end for**
9: Return final recommendation

How We Evaluate Methods

There are really **three** layers to evaluation:

1. **Decision quality:** did the method identify a good move?
2. **Allocation behavior:** did it spend budget in the right places?
3. **Proxy-truth validity:** is the reference strong enough to support the comparison?

A method can look bad for at least three different reasons:

- the state is intrinsically hard,
- the proxy truth is still ambiguous or unstable,
- the method spent its rollouts poorly.

Notation for Evaluation Against Proxy Truth

For one local decision problem, let:

$$A = \text{candidate move set}, \qquad a^\star = \arg\max_{a \in A} \mu_{\mathrm{ref}}(a).$$

$$\hat{a}(b) = \text{move recommended at checkpoint } b, \qquad \mathrm{rank}_{\mathrm{ref}}(a) = \text{proxy-truth rank of } a.$$

$$n_b(a) = \text{rollouts allocated to move } a \text{ by budget } b, \qquad B = \sum_{a \in A} n_b(a).$$

Important point

All one-run performance metrics are evaluated against the *same* proxy-truth object for that state or opening roll.

Primary Decision Metrics

The main one-run decision metrics are all defined against the proxy truth:

$$\texttt{top1\_match} = \mathbf{1}\{\hat{a}(b) = a^\star\}, \qquad \texttt{truth\_top2\_hit} = \mathbf{1}\{\mathrm{rank}_{\mathrm{ref}}(\hat{a}(b)) \le 2\}.$$

$$\texttt{selected\_reference\_rank} = \mathrm{rank}_{\mathrm{ref}}(\hat{a}(b)), \qquad \texttt{simple\_regret} = \mu_{\mathrm{ref}}(a^\star) - \mu_{\mathrm{ref}}(\hat{a}(b)).$$

Interpretation:

- `top1_match` asks whether the winner is exactly correct.
- `simple_regret` asks how costly the recommendation is when it is wrong.
- `selected_reference_rank` is often less brittle than a pure 0/1 success indicator.

Primary Allocation Metrics

The key allocation metrics tie the rollout path back to the proxy truth:

$$\texttt{share\_best\_truth} = \frac{n_b(a^\star)}{B}, \qquad \texttt{share\_top2\_truth} = \frac{\sum_{a \in \mathrm{Top2}_{\mathrm{ref}}} n_b(a)}{B}.$$

If S is the set of moves screened as clearly inferior by the truth object, then

$$\texttt{share\_mc\_screened\_suboptimal} = \frac{\sum_{a \in S} n_b(a)}{B}.$$

A stronger waste metric is

$$\texttt{gap\_weighted\_wasted\_allocation} = \sum_{a \in A} \frac{n_b(a)}{B} \left(\mu_{\mathrm{ref}}(a^\star) - \mu_{\mathrm{ref}}(a)\right).$$

High share to the truth top two is usually good; high waste is usually bad.

Hardness and Reference Quality

A method should not be judged without understanding the problem itself.
Important context quantities:

$$\texttt{top\_two\_gap\_estimate} = \mu_{\mathrm{ref}}(\mathrm{rank}\,1) - \mu_{\mathrm{ref}}(\mathrm{rank}\,2).$$

$$\texttt{mc\_gap\_excludes\_zero} = \mathbf{1}\{\text{approximate top-two gap interval excludes } 0\}.$$

We also use:

- `n_near_optimal`: how many moves are essentially tied near the top,
- `mc_not_separated_from_best_set_size`: how many moves are still not Monte Carlo-separated from the best.

Large gap means an easier problem; a crowded top of the truth table means a harder one.

Per-Move Proxy-Truth Precision

For each move a, after $n(a)$ reference rollouts, the truth object stores:

$$\texttt{reference\_mean}(a) = \frac{\texttt{reward\_sum}(a)}{n(a)}, \qquad \texttt{reference\_se}(a) = \sqrt{\frac{\texttt{sample\_variance}(a)}{n(a)}}.$$

A simple Monte Carlo precision interval is:

$$\texttt{reference\_mean}(a) \pm 1.96\,\texttt{reference\_se}(a).$$

We also track

$$\texttt{unresolved\_fraction}(a) = \frac{\texttt{unresolved}(a)}{n(a)}.$$

Important interpretation

These are *Monte Carlo precision intervals* for the proxy truth. They are not Bayesian credible intervals and they are not external truth intervals for real backgammon value.

Top-of-Truth Separation Metrics

The most important hardness quantity is the gap between the best and second-best truth moves:

$$\texttt{top\_two\_gap\_estimate} = \mu_{\mathrm{ref}}(\mathrm{rank}\,1) - \mu_{\mathrm{ref}}(\mathrm{rank}\,2).$$

An approximate Monte Carlo standard error is

$$\texttt{top\_two\_gap\_se} \approx \sqrt{\texttt{reference\_se}(\mathrm{rank}\,1)^2 + \texttt{reference\_se}(\mathrm{rank}\,2)^2}.$$

This yields an approximate interval:

$$\texttt{top\_two\_gap\_estimate} \pm 1.96\,\texttt{top\_two\_gap\_se}.$$

Validity note

This is an approximation. It ignores covariance between the top two estimates, so the package treats it as a screening device rather than proof.

Truth-Side Screening and Ambiguity

For any move a, define its gap below the best move as

$$\texttt{gap\_to\_best}(a) = \mu_{\mathrm{ref}}(a^\star) - \mu_{\mathrm{ref}}(a).$$

A move is marked near-optimal if

$$\texttt{near\_optimal}(a) = \mathbf{1}\{\texttt{gap\_to\_best}(a) \le \varepsilon_{\mathrm{near}}\}.$$

A move is marked as not clearly separated from the best if its Monte Carlo upper bound still overlaps the best move's lower bound.
We summarize this by:

- `n_near_optimal`
- `mc_not_separated_from_best_set_size`

These tell us whether the top of the truth ranking is genuinely crowded or well-separated.

Certification and Stability Labels

The package uses summary labels to describe the *quality of the proxy truth*, not just the performance of the algorithms.

- `certification`: labels such as clear, ambiguous, hard, uncertain
- `reference_stability_label`: whether repeated truth builds stabilize
- `decision_screen_label`: whether the top decision looks screen-clear or still ambiguous

Why this matters

A method comparison is much more convincing when the proxy truth is itself stable and the best-vs-second-best decision is clearly separated.

Posterior Confidence Versus Proxy Truth

The package also reports model-relative posterior confidence, for example:

$$\texttt{recommended\_prob\_best} = \mathbb{P}_{\text{model}}(\hat{a}(b) \text{ is best} \mid \text{data up to } b).$$

Operationally, this is estimated by repeated posterior sampling.
This is useful, but it is **not** itself a truth metric. High confidence is only reassuring when it comes together with:

- low simple regret,
- good truth rank,
- and a well-separated proxy truth.

So posterior confidence must always be interpreted relative to the proxy reference.

How Proxy Truth Supports Method Validity

A method comparison is most defensible when several things line up:

1. the proxy-truth top-two gap is not tiny,
2. the approximate gap interval excludes zero,
3. only a small number of moves are near-optimal,
4. repeated truth builds look stable,
5. the method's posterior confidence agrees with low regret.

In other words, validity comes from combining:

- method-side metrics,
- truth-side precision,
- and truth-side stability.

What a Strong Result Looks Like

A strong comparative result does *not* just say:

Weak interpretation

Method A has a slightly better average number than Method B.

A strong result says:

Strong interpretation

Method A performs better because it allocates more budget to the top contenders and less budget to clearly weak moves, especially in hard problems with small top-two gaps.

So the presentation should tie together:

1. final decision quality,
2. allocation behavior,
3. problem hardness.

What Results Should Be Reported

Even without pasting graphs here, the presentation should eventually report results in the following structure:

1. **Headline comparison**
   - TS vs TTTS vs other TS variants,
   - top1_match and simple_regret across budgets.
2. **Mechanism**
   - share_top2_truth,
   - share_mc_screened_suboptimal,
   - gap-weighted wasted allocation.
3. **Hardness and validity context**
   - top-two gap and near-optimal count,
   - whether proxy truth clearly separates the top contenders,
   - whether the truth object itself appears stable and screen-clear.

How to Compare the Models

Model comparison should be done carefully.
Correct design:

- fix one algorithm,
- change only the model,
- compare how:
  - the recommended move changes,
  - the allocation changes,
  - the confidence changes.

This means comparisons such as:

- TS + Beta–Bernoulli,
- TS + Dirichlet–Categorical,
- TS + Student-t.

The point is not to declare a universal winner. The point is to understand *when model choice matters*.

How to Compare the Algorithms

Algorithm comparison should also be controlled.
Correct design:

- fix one model,
- compare TS variants under the same problem, truth reference, and budget.

This lets us ask:

- does TTTS help because it focuses more on the top competition?
- does soft elimination reduce waste?
- does forced exploration prevent early lock-in?
- does top-k focus help or accidentally freeze out the truth-best move?

Final Takeaway

This project is not just about running many simulations.
It is about understanding:

- how uncertainty is modeled,
- how posterior information is converted into rollout allocation,
- how that allocation affects final decision quality under finite budgets.

The three model classes answer different scientific questions:

- Beta–Bernoulli: win probability,
- Dirichlet–Categorical: scored outcome structure,
- Student-t: scalar expected payoff.

The Thompson variants answer different allocation questions: baseline exploration, top-two competition, repeated posterior voting, elimination of weak moves, protection against premature lock-in, and focus on promising subsets.

Conclusion

The central message is:

Main message

Finite-budget success depends not only on estimating move values, but on spending rollouts where they matter most.

So the project studies the interaction between:

1. Bayesian uncertainty modeling,
2. sequential allocation rules,
3. the hardness structure of local backgammon decision problems.

That is the statistical heart of the package.